Expt. No. → Student Management using JDBC.   Date: 6 April, 2026   P. No.: 39

## Challenging Task

Aim:- To develop a Student Management System using JDBC to perform database operation like insert, update, delete and display records.

Objectives:-

To understand JDBC connectivity
To perform CRUD operations.
To connect java with MySQL database.
To manage student records efficiently.

Concepts Used
JDBC (Java Database Connectivity)
MySQL database
SQL Queries (INSERT, UPDATE, DELETE, SELECT)
OOP concepts in java.

Theory

JDBC is an API that allows java programs to interact with databases. It provides methods to query & update data in a database using SQL.

# Abstract

This project is based on developing a Student Management System using Java Database Connectivity. This system allows users to connect java application with MySQL database to manage student info. efficiently.
It supports basic CRUD operations such as inserting new records, updating existing records, deleting records and displaying student details.

Steps in JDBC:

1. Load Driver
2. Establish connection
3. Create Statement
4. Execute Query
5. Close Connection

Advantages:
→ Platform Independent
→ Secure data handling
→ Efficient database operations

Algorithm

1. Start
2. Load JDBC driver
3. Establish connection
4. Take user input.
5. Perform operation (Insert / update / Delete / View)
6. Display result
7. Close connection
8. End.

9. Source code

| ID | Name | Age | Course |
| --- | --- | --- | --- |
| 1 | Aman | 20 | BCA |

Expt. No.

Date :
P. No. : 41

9. Source Code

```
import java.sql.*;

public class StudentDB {
    public static void main (String[] args) {
        try {
            Connection con = DriverManager.getConnection(
              "jdbc:mysql://localhost:3306/studentdb",
                "root", "password");
    Statement stmt = con.createStatement();
    stmt.executeUpdate("Insert into student Values(
                 1,'Rahul', 20, 'CSE')");

  Result rs = stmt.executeQuery("Select * from student");
  while (rs.next()){
      System.out.println(rs.getInt(1) + " " +
                  rs.getString(2) + " " +
                  rs.getInt (3) + " " +
                  rs.getString (4));
  }
```

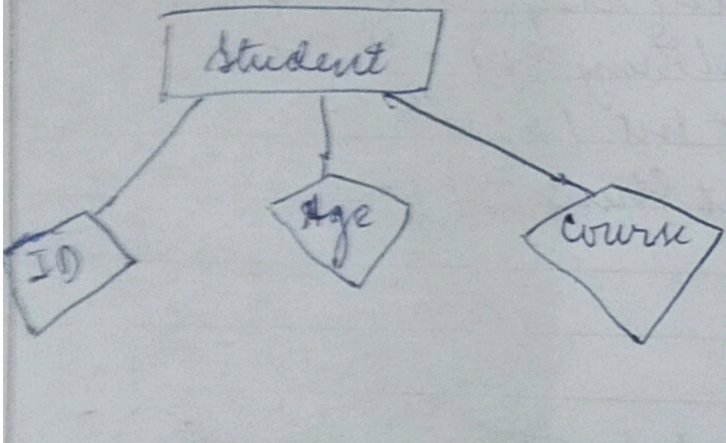
~~Result~~
~~The student management system was successfully implemented using JDBC and database operations were performed efficiently.~~

Teacher's Signature

Databan Design

| S.No | Name of the Feild | Data Type | Size | Constraint | Description |
|---|---|---|---|---|---|
| 1 | ID | INT | 10 | Primary key | Unique Student ID |
| 2 | Name | VARCHAR | 50 | Not Null | Student Name |
| 3 | Age | INT | 3 | Not Null | Student Age |
| 4 | Course | VARCHAR | 50 | | Course Name |

ER Diagram :-



Expt No.

## 6) System Specification

Hardware Requirement
- Processor : Intel i3 or above
- RAM : Minimum 4GB
- Hard Disk : 500GB

Software Requirement
- Operating System : Window 10/11
- Programming Language : java
- Database : MYSQL
- IDE : Eclipse / Netbeans
- JDBC Driver

## 7) Testing Striategies :-

Unit Testing :- Individual functions tested seperately

Integrated Testing :- Java and database connectivity tested

System Testing :- Entire application tested as a whole.

Output Testing :- Verified correctness of results.

Conclusion :-

The Student Management System was created successfully implemented using JDBC. The project helped in understanding database connectivity & performing CRUD operations efficiently using java.

Future Enhancement :-

- Add GUI interface using Swing.
- Add login Authentication system.
- Store more details like address and contact
- Implement search and filter options.

References :-


- Java documentations
- MySQL Documentations
- JDBC API guide
- Programming Notes / Classroom Material